TECNICATURA UNIVERSITARIA EN DESARROLLO WEB · FCAD · UNER

Unidad 2
TypeScript, React y ciclo de vida

Desarrollo para Móviles · 2026 — Segundo cuatrimestre

Prof. Titular: Lic. Franco Ruis Dias · JTP: Lic. Germán Cueva

Expo SDK 57 · React Native 0.86 · React 19.2

UNIDAD 2

Qué vamos a ver

Objetivos

- Repasar TypeScript y React con la mirada puesta en móvil.
- Escribir componentes tipados y reutilizables.
- Entender **cuándo** se ejecuta cada parte de un componente.

Temas

- TypeScript vs. JavaScript
- Componentes, props y estado
- Hooks: los que vas a usar todos los días
- **Ciclo de vida: componente, pantalla y app**

Esta es la **primera clase de la Unidad 2**. La segunda —React Native, arquitectura de ejecución y Expo— arranca donde termina ésta.

UNIDAD 2 · PARTE 1

TypeScript

Qué agrega sobre JavaScript y por qué en móvil pesa más que en web.

Qué es

Un **superconjunto de JavaScript**: todo JavaScript válido es TypeScript válido. Le agrega un sistema de tipos que se verifica antes de ejecutar, y después desaparece — al dispositivo llega JavaScript común.

Tipado estático
Declarás qué tipo tiene cada variable, parámetro y retorno.

Se borra al compilar
Los tipos no existen en tiempo de ejecución: no hay costo de performance.

Gradual
Podés tipar un archivo y dejar el resto como está. No es todo o nada.

> El template de `create-expo-app` ya viene en TypeScript. No hay que configurar nada para empezar.

Por qué importa más en móvil

En web

Un error de tipos rompe la página. El usuario recarga con F5 y seguís. Si el bug llegó a producción, subís el arreglo y en minutos está.

En una app

Un error de tipos **cierra la aplicación**. Y el arreglo tiene que pasar por la tienda: días de revisión, más los usuarios que nunca actualizan.

> El costo de un error crece con lo lejos que llega. TypeScript lo corre hacia el momento más barato posible: mientras escribís.

Tipos e inferencia

ANOTADO A MANO

```ts
let nombre: string = 'Ana';
let edad: number = 24;
let activo: boolean = true;
let etiquetas: string[] = ['nuevo'];

function saludar(a: string): string {
  return 'Hola ' + a;
}
```

INFERIDO POR TYPESCRIPT

```ts
let nombre = 'Ana';        // string
let edad = 24;             // number
let activo = true;         // boolean
let etiquetas = ['nuevo']; // string[]

function saludar(a: string) {
  return 'Hola ' + a;      // devuelve string
}
```

> No hace falta anotar todo. Lo que sí conviene anotar siempre: los **parámetros** de una función y la forma de los datos que vienen de afuera (una API, por ejemplo), porque ahí TypeScript no tiene de dónde inferir.

Interfaces: la forma de un objeto

```ts
interface Producto {
  id: number;
  nombre: string;
  precio: number;
  descripcion?: string;   // el ? lo hace opcional
}

interface PropsTarjeta {
  producto: Producto;
  alTocar: (id: number) => void;
}

function Tarjeta({ producto, alTocar }: PropsTarjeta) {
  // ...
}
```

> Es el uso número uno de TypeScript en React: **describir las props** de un componente. Escribir la interfaz es documentar el componente y validarlo al mismo tiempo.

Uniones: cuando hay varios estados posibles

```ts
type Estado =
  | { fase: 'cargando' }
  | { fase: 'listo'; productos: Producto[] }
  | { fase: 'error'; mensaje: string };

function render(estado: Estado) {
  if (estado.fase === 'cargando') return 'Cargando…';
  if (estado.fase === 'error') return estado.mensaje;
  return estado.productos.length + ' productos';
  // acá TypeScript ya sabe que .productos existe
}
```

> Los tres estados de una pantalla que pide datos —cargando, listo, error— son un caso perfecto para una unión. El compilador no te deja olvidarte de uno.

Genéricos: el tipo como parámetro

```
// T se decide en cada llamada, no al escribir la función
async function traer<T>(url: string): Promise<T> {
  const respuesta = await fetch(url);
  return respuesta.json();
}

const productos = await traer<Producto[]>('/api/productos');
// productos es Producto[], no "any"

const usuario = await traer<Usuario>('/api/perfil');
// usuario es Usuario
```

> Sin genéricos habría que escribir una función por cada tipo de dato, o resignarse a `any` y perder toda la ayuda del editor.

UNIDAD 2 · PARTE 2

React: componentes y props

La unidad mínima con la que se construye una interfaz.

La idea central

React es **declarativo**: no describís los pasos para cambiar la pantalla, describís cómo tiene que verse la pantalla para un estado dado. React se encarga del resto.

IMPERATIVO

```
const el = document.querySelector('#total');
el.textContent = carrito.length;
if (carrito.length === 0) {
  el.classList.add('vacio');
} else {
  el.classList.remove('vacio');
}
```

DECLARATIVO

```
<Text style={carrito.length === 0
  ? estilos.vacio
  : estilos.normal}>
  {carrito.length}
</Text>
```

> En el segundo caso no hay que acordarse de sacar la clase. **El estado es la única fuente de verdad** y la interfaz es una función de ese estado.

Un componente es una función

```jsx
import { View, Text } from 'react-native';

export default function TarjetaPerfil() {
  return (
    <View>
      <Text>Ana Gómez</Text>
    </View>
  );
}
```

PascalCase

`TarjetaPerfil`, no `tarjetaPerfil`. Con minúscula React lo trata como etiqueta común.

Devuelve JSX

Un solo elemento raíz. Si necesitás varios hermanos, van dentro de un `<>…</>`.

Se anidan

Un componente puede usar otros adentro. Así se arma toda la pantalla.

JSX y TSX

LO QUE ESCRIBÍS

```
<View style={estilos.caja}>
  <Text>Hola {nombre}</Text>
</View>
```

LO QUE CORRE

```
jsx(View, { style: estilos.caja,
  children: jsx(Text, {
    children: ['Hola ', nombre]
  })
});
```

No es HTML

Es azúcar sintáctico para llamadas a función. Por eso las llaves `{ }` aceptan cualquier expresión de JavaScript.

`.tsx`, no `.ts`

Un archivo con JSX y TypeScript lleva extensión `.tsx`. Es la que vas a usar para todo componente.

Props: de padre a hijo

```tsx
interface Props {
  nombre: string;
  edad?: number;          // opcional
}

function Perfil({ nombre, edad = 0 }: Props) {
  return <Text>{nombre} ({edad})</Text>;
}

// desde el padre
<Perfil nombre="Ana" edad={24} />
```

> Las props son de **sólo lectura**. Un componente nunca modifica las props que recibe: si necesita cambiar algo, avisa al padre.

De hijo a padre: pasar una función

El hijo avisa

```tsx
type Item = { id: number; nombre: string };

interface Props {
  item: Item;
  aBorrar: (id: number) => void;
}

function Fila({ item, aBorrar }: Props) {
  return (
    <View>
      <Text>{item.nombre}</Text>
      <Button title="Borrar"
              onPress={() => aBorrar(item.id)} />
    </View>
  );
}
```

El padre decide

```tsx
const INICIALES: Item[] = [
  { id: 1, nombre: 'Yerba' },
  { id: 2, nombre: 'Bizcochos' },
  { id: 3, nombre: 'Azúcar' },
];

function Lista() {
  const [items, setItems] = useState(INICIALES);
  const borrar = (id: number) =>
    setItems(xs => xs.filter(x => x.id !== id));

  return (
    <View>
      {items.map(item => (
        <Fila key={item.id} item={item}
              aBorrar={borrar} />
      ))}
    </View>
  );
}
```

> Los datos bajan, los eventos suben. `Fila` no sabe qué va a pasar: sólo avisa que apretaron el botón, con el `id` de su propio ítem.

Estado vs. props

Props

- Vienen de afuera, las manda el padre.
- Sólo lectura.
- Cambian cuando el padre las cambia.

Estado

- Vive adentro del componente.
- Se cambia con su función `set…` .
- Cambiarlo dispara un nuevo renderizado.

> Regla práctica: si un dato se puede calcular a partir de las props o de otro estado, **no lo guardes en estado**. Calculalo en el renderizado. Dos fuentes de verdad para el mismo dato terminan siempre desincronizadas.

Listas y `key`

```
{productos.map((producto) => (
  <Tarjeta key={producto.id} producto={producto} />
))}

// ❌ el índice no sirve como key
{productos.map((producto, i) => (
  <Tarjeta key={i} producto={producto} />
))}
```

La `key` tiene que ser **estable y propia del dato** —el `id`, no el índice del arreglo. Con el índice, al borrar un elemento del medio React reutiliza el componente equivocado: el estado interno queda pegado a la fila que no corresponde.

Buenas prácticas

Componentes chicos

Uno solo propósito. Si la función no entra en pantalla, hay más de un componente adentro.

Nombres que dicen qué es

`TarjetaProducto`, no `Item2`.

Props siempre tipadas

Una `interface Props` por componente. Es la documentación que no se desactualiza.

Sin lógica de negocio en el JSX

Calculá arriba, mostrá abajo. El `return` se lee de un vistazo.

UNIDAD 2 · PARTE 3

Hooks

Cómo un componente que es una función puede recordar cosas.

El problema que resuelven

Un componente es una función que se ejecuta entera cada vez que hay que redibujar. Todas sus variables locales se pierden. Entonces, ¿dónde guarda lo que tiene que recordar entre un renderizado y el siguiente?

Antes: clases REACT < 16.8

El estado vivía en `this.state`, y había que lidiar con `this`, `bind` y métodos de ciclo de vida.

Ahora: hooks

React guarda el estado por afuera del componente y se lo devuelve en cada llamada. La función queda limpia.

> Un hook es una función que empieza con `use` y que le pide algo a React. Vas a usar los que trae React, los que trae Expo, y los que escribas vos.

Las dos reglas

1 · Siempre en el nivel superior

Nunca dentro de un `if`, un bucle o una función anidada.

```
// ❌
if (hayUsuario) {
  const [n, setN] = useState(0);
}
```

2 · Sólo desde componentes o hooks propios

No desde una función común, un manejador de evento o una clase.

```
// ✅
const [n, setN] = useState(0);
if (hayUsuario) { /* usar n */ }
```

> React identifica cada hook **por el orden** en que se llama, no por su nombre. Si el orden cambia entre dos renderizados, el estado se mezcla.

useState

```
const [cantidad, setCantidad] = useState(0);
const [productos, setProductos] = useState<Producto[]>([]);

// ❌ los dos leen el mismo valor viejo: suma 1, no 2
setCantidad(cantidad + 1);
setCantidad(cantidad + 1);

// ✅ actualización en base al valor previo
setCantidad(c => c + 1);
setCantidad(c => c + 1);
```

`cantidad` es una **constante dentro de ese renderizado**: no cambia de valor por llamar a `setCantidad`. El valor nuevo se ve recién en el renderizado siguiente.

useEffect

```
useEffect(() => {
  const id = setInterval(() => console.log('tic'), 1000);

  return () => clearInterval(id);   // limpieza
}, []);
```

`[]`
Se ejecuta una vez, al montarse.

`[a, b]`
Se vuelve a ejecutar cuando cambia `a` o `b` (limpiando antes).

sin arreglo
Se ejecuta después de cada renderizado. Casi nunca es lo que querés.

> Sin la función de limpieza, ese `setInterval` queda vivo para siempre. En una app que el usuario deja abierta días, eso se acumula.

useContext

```
const TemaContexto = createContext<'claro' | 'oscuro'>('claro');

function Raiz() {
  return (
    <TemaContexto value="oscuro">
      <Pantalla />
    </TemaContexto>
  );
}

function BotonProfundo() {
  const tema = useContext(TemaContexto);   // sin pasar props en cadena
}
```

> Para datos que necesita medio árbol: tema, usuario logueado, idioma. Se retoma en la Unidad 3 (temas) y en la Unidad 5 (estado global).

useRef

```
const cantidadRenderizados = useRef(0);
cantidadRenderizados.current += 1;   // NO dispara un renderizado

const campo = useRef<TextInput>(null);
<TextInput ref={campo} />
<Button title="Enfocar" onPress={() => campo.current?.focus()} />
```

Una caja que persiste

El valor sobrevive entre renderizados, pero cambiarlo no redibuja nada.

Referencia a un componente

Para operaciones imperativas que no se pueden expresar con props: `focus()`, `scrollTo()`, reproducir un video.

> Si el valor se muestra en pantalla, va en `useState`, no en `useRef`: la interfaz no se va a enterar del cambio.

`useMemo` y `useCallback`

```tsx
// memoriza un VALOR: no recalcula si no cambian las dependencias
const ordenados = useMemo(
  () => productos.slice().sort((a, b) => a.precio - b.precio),
  [productos]
);

// memoriza una FUNCIÓN: mantiene la misma identidad entre renderizados
const alTocar = useCallback((id: number) => abrir(id), []);
```

No los pongas por las dudas. Memorizar tiene su propio costo. Van cuando hay un cálculo caro medido, o cuando la función se pasa como dependencia de un efecto o a un componente memorizado — como en `useFocusEffect`, que vemos en un rato.

React Compiler ESTABLE DESDE V1.0

Es un compilador que **memoriza por vos**: analiza el componente en tiempo de build y agrega la memorización donde hace falta. Se activa con una línea.

Prenderlo

```
{ "expo": {
    "experiments": { "reactCompiler": true }
} }
```

Desde SDK 54 Babel ya viene configurado: alcanza con el flag. Las reglas de ESLint vienen incluidas en `eslint-config-expo` desde SDK 55.

Qué cambia en tu código

Nada, y ese es el punto. Podés borrar los `useMemo`, `useCallback` y `React.memo` que estaban ahí sólo por rendimiento.

Si un componente se rompe, se lo excluye con la directiva `'use no memo'` adentro de la función.

Sólo optimiza código que **respeta las reglas de React**: nada de mutar props o estado durante el render. `npx react-compiler-healthcheck` revisa el proyecto antes de prenderlo. Los componentes de clase quedan afuera.

useReducer

```tsx
type Accion = { tipo: 'sumar' } | { tipo: 'restar' } | { tipo: 'reiniciar' };

function reducer(estado: number, accion: Accion): number {
  switch (accion.tipo) {
    case 'sumar':     return estado + 1;
    case 'restar':    return estado - 1;
    case 'reiniciar': return 0;
  }
}

const [cantidad, despachar] = useReducer(reducer, 0);
<Button title="+" onPress={() => despachar({ tipo: 'sumar' })} />
```

> Cuando el estado tiene varias transiciones repartidas en muchos manejadores, conviene juntar toda la lógica de actualización en una función afuera del componente.

Hooks propios

```
// hooks/useContador.ts
export function useContador(inicial = 0) {
  const [valor, setValor] = useState(inicial);
  const sumar    = useCallback(() => setValor(v => v + 1), []);
  const reiniciar = useCallback(() => setValor(inicial), [inicial]);
  return { valor, sumar, reiniciar };
}

// en cualquier pantalla
const { valor, sumar } = useContador();
```

Un hook propio es sólo una función que empieza con `use` y usa otros hooks adentro. Es la forma de **reutilizar lógica** —no interfaz— entre componentes.

Lo que trajo React 19

SDK 57 usa **React 19.2**. Tres cambios que se notan apenas empezás a escribir componentes.

```tsx
// 1 · ref es una prop más: forwardRef ya no hace falta
function Campo({ etiqueta, ref }: { etiqueta: string; ref?: Ref<TextInput> }) {
  return <TextInput ref={ref} placeholder={etiqueta} />;
}

// 2 · el Context se usa directo como proveedor
<TemaContext value={tema}>{hijos}</TemaContext>   // antes: .Provider

// 3 · use() lee un contexto (o una promesa) y puede ir adentro de un if
const tema = use(TemaContext);
```

> `use` es la única excepción a la primera regla de los hooks: **sí puede llamarse condicionalmente**. Las demás siguen igual.

React 19.2: `useEffectEvent` y `Activity`

USEEFFECTEVENT

```
const avisar = useEffectEvent(() => {
  mostrarToast(`Conectado a ${sala}`, tema);
});

useEffect(() => {
  const c = conectar(sala);
  c.on('listo', avisar);
  return () => c.cerrar();
}, [sala]);   // tema NO va acá
```

Lee siempre el valor último de `tema`, pero no re-dispara el efecto cuando cambia.

<ACTIVITY>

```
<Activity mode={visible ? 'visible' : 'hidden'}>
  <Formulario />
</Activity>
```

Esconde la pantalla **sin desmontarla**: se desmontan los efectos, pero el estado —lo que el usuario ya escribió— queda.

> Los dos atacan el mismo dolor: el arreglo de dependencias del efecto. Uno saca de ahí lo que no debería disparar nada; el otro evita el `{'{'}visible && <Formulario />{'}'}` que borra todo lo escrito.

UNIDAD 2 · PARTE 4

Ciclo de vida

Cuándo se ejecuta cada cosa — y por qué en móvil hay tres relojes, no uno.

UNIDAD 2 · PARTE 4 · CICLO DE VIDA

Los tres momentos de un componente

Montaje
React ejecuta la función por primera vez y pone el resultado en pantalla.

Actualización
Cambian las props o el estado: la función se vuelve a ejecutar entera y React aplica sólo la diferencia.

Desmontaje
React lo saca de la pantalla. Lo que el componente haya dejado abierto hay que cerrarlo acá.

El componente tiene ciclo de vida. Los **hooks no son métodos de ciclo de vida**: son la herramienta con la que reaccionás a esos momentos.

Cómo se hacía antes

CLASE CÓDIGO HEREDADO

```
class Reloj extends React.Component {
  componentDidMount() {
    this.id = setInterval(this.tic, 1000);
  }
  componentWillUnmount() {
    clearInterval(this.id);
  }
  render() { /* … */ }
}
```

FUNCIÓN LO QUE USAMOS

```
function Reloj() {
  useEffect(() => {
    const id = setInterval(tic, 1000);
    return () => clearInterval(id);
  }, []);
  // …
}
```

Lo que estaba partido en dos métodos lejanos —abrir arriba, cerrar abajo— queda junto. Esa cercanía es la razón principal del cambio: es muy difícil olvidarse de limpiar algo que está tres líneas más abajo.

De los métodos de clase a los hooks

Método de clase	Equivalente con hooks
`componentDidMount`	`useEffect(() => {…}, [])`
`componentDidUpdate`	`useEffect(() => {…}, [dependencias])`
`componentWillUnmount`	el `return` del `useEffect`
`this.state` / `setState`	`useState`
`shouldComponentUpdate`	`React.memo`

> La tabla sirve para **leer código viejo**, no para escribir código nuevo. No es una traducción exacta, y pensar en efectos como "métodos de montaje" lleva derecho a poner mal las dependencias.

El efecto tiene su propio ciclo

En vez de pensar «esto corre al montar», pensá: **empezar a sincronizar** y **dejar de sincronizar**.

```
useEffect(() => {
  const sub = suscribirseA(salaId);      // empezar a sincronizar
  return () => sub.cancelar();           // dejar de sincronizar
}, [salaId]);
```

> Cuando `salaId` cambia de `'a'` a `'b'`, React **primero cancela** la suscripción a `'a'` y recién después se suscribe a `'b'`. No hace falta escribir esa coreografía: alcanza con decir cómo se empieza y cómo se termina.

Por qué se ejecuta dos veces en desarrollo

Si envolvés la app en `<StrictMode>`, React monta cada componente, lo desmonta y lo vuelve a montar — una sola vez, y sólo en desarrollo.

No es un bug

Es a propósito: fuerza a que se ejecute la limpieza. Un efecto bien escrito no nota la diferencia.

Es un detector

Si ves dos requests, dos suscripciones o el contador saltando de a dos, **falta la limpieza**.

> En la build de producción esto no pasa. Si "arreglás" el síntoma sacando `StrictMode`, el bug sigue estando: sólo dejaste de verlo.

Segundo reloj: la pantalla

En web, al navegar de A a B, A se desmonta. **En una app no.** React Navigation deja la pantalla montada para poder volver instantáneamente y conservar el scroll, el formulario a medio llenar y la posición de la lista.

> Consecuencia directa: al salir de una pantalla **no corre la limpieza** de sus efectos, y al volver **no se vuelven a ejecutar**. El `useEffect(…, [])` que refresca datos "al entrar" corre una sola vez en toda la vida de la app.



Inicio sigue montada, abajo

useFocusEffect

```
import { useFocusEffect } from 'expo-router';
import { useCallback } from 'react';

export default function Perfil() {
  useFocusEffect(
    useCallback(() => {
      const id = setInterval(refrescarSaldo, 5000);
      return () => clearInterval(id);     // al perder el foco
    }, [])
  );
}
```

Es `useEffect` atado al **ciclo de vida de la navegación** en lugar del componente: corre cuando la pantalla toma el foco y limpia cuando lo pierde. El `useCallback` no es opcional — sin él la función es nueva en cada renderizado y el efecto se reinicia sin parar.

Tercer reloj: la app

```
import { AppState } from 'react-native';

useEffect(() => {
  const sub = AppState.addEventListener('change', (estado) => {
    if (estado === 'active') refrescar();   // volvió del segundo plano
  });
  return () => sub.remove();
}, []);
```

`active`
En primer plano. El usuario la está usando.

`background`
En otra app, o en la pantalla de inicio.

`inactive` iOS
De paso entre las dos, o durante una llamada entrante.

Los tres relojes

Componente

Montaje · actualización · desmontaje

`useEffect`

Pantalla

Foco · pérdida de foco

`useFocusEffect`

Aplicación

Primer plano · segundo plano

`AppState`

> Los tres corren al mismo tiempo y son independientes. La pregunta a hacerse antes de escribir un efecto es **cuál de los tres** es el que corresponde.

Los cuatro errores de siempre

Efecto sin limpieza

Temporizadores, suscripciones y listeners que quedan vivos. Se acumulan con cada montaje.

Sacar dependencias del arreglo

Para "que no se ejecute de más". El efecto queda leyendo valores viejos. Si sobra una dependencia, revisá el efecto.

Esperar que la pantalla se remonte

No pasa. Si tiene que correr cada vez que se entra, va `useFocusEffect`.

`useEffect` para derivar estado

Un efecto que sólo hace `setEstado` a partir de props sobra: calculalo durante el renderizado.

UNIDAD 2

Lo que sigue

- **Segunda clase de la Unidad 2:** React Native, arquitectura de ejecución y Expo.
- El ciclo de vida de la pantalla vuelve en la **Unidad 3** con Expo Router, y el de la app en la **Unidad 4** con las APIs del dispositivo.

Documentación: `react.dev` · `typescriptlang.org/docs` · `reactnavigation.org`